DIM0501 — Boas Práticas de Programação

Relatório do Projeto da Unidade 1 — AgendaFácil

Modelo de entrega — serve para Python, Java e C++

Prof. Marcus Terra · DIMAp/UFRN · Semestre 2026.1 · Entrega: 25/10/2026

Integrantes (nome e matrícula)	Linguagem
Ian Silva Cruz de Medeiros (20240022663)	Python
Repositório ou arquivo entregue	
ian_silva_projeto_unidade1_boas_praticas.zip	

1. Diagnóstico do código-base

#	Categoria	Anomalia e onde está	Por que é um problema
1	Adequação e lógica	Linha 22 - `ag.append([p, pr, d, h, dur, "MARCADO"])`	Guardar todos os dados em uma única lista, gerando dificuldade de acesso
2	Adequação e lógica	Variável - `ok = True`	Variável desnecessária para realizar a lógica condicional
3	Manutenibilidade	Variáveis - `p, pr, d, h, dur`	Nome não permite entender o que significa
4	Manutenibilidade	Linha 22 - `ag.append([p, pr, d, h, dur, "MARCADO"])`	“MARCADO” é guardado junto da lista de agendamentos, apenas para ser exibido posteriormente na função list(), misturando responsabilidades
5	Manutenibilidade	Variável - `ag = []`	É acessada de forma global por todas as funções, gerando complexidade e dificuldade de interpretação. Pois as responsabilidades não ficam claramente divididas, as funções gostariam de ser uma classe.

6	Organização e estética	`# adiciona na lista`	Comentário repetitivo usado para substituir código auto-explicativo
7	Organização e estética	`# lista os atendimentos do mes`	Comentário enganoso na função list(d), o comportamento real da função é listar o dia
8	Programação defensiva e segurança	Função - `add(p, pr, d, h, dur)`	Nenhuma validação é feita para `p, pr, d`
9	Programação defensiva e segurança	Função - `add(p, pr, d, h, dur)`	Exceções são engolidas com “except: pass”
10	Desempenho e recursos	Linha 35 - `for b in ag:`	Há 2 loops aninhados dentro da função de “remarcar” de maneira desnecessária

2. O que foi resolvido e o que ficou como dívida

#	Situação	Como foi resolvida, ou por que ficou
1	resolvido	Dados do agendamento são guardados em atributos nomeados de uma classe.
2	resolvido	Lógica condicional é feita por funções de validação que retornam True ou False.
3	resolvido	Variáveis renomeadas para ‘paciente’, ‘medico’, ‘data’, ‘hora’, ‘duracao’.
4	resolvido	“MARCADO” é exibido por um print().
5	resolvido	A lista de atendimentos é o atributo de uma classe acessada por seus métodos.
6	resolvido	Comentário removido.
7	resolvido	Comentário resolvido.
8	resolvido	Argumentos nulos e valores não aceitos pelas regras de negócio são tratados.
9	resolvido	Exceções são tratadas explicitamente e exceções não são ‘engolidas’.
10	resolvido	“Remarcar” obtêm os dados do agendamento atual e do agendamento pretendido e não executa desnecessariamente o loop.

3. Decisões de projeto

Decisão tomada	Alternativa descartada	Por que decidimos assim
Uso de constantes globais para regras de negócio de horário e duração.	Uso de constantes locais nas variáveis responsáveis por sua validação.	Um único local com constantes globais para regras de negócio facilita a manutenção.
A validação de data trata formatos fora de AAAA-MM-DD, mas trata apenas parcialmente datas inexistentes (ex.: 31 de fev.)	Uso de regex e calendário completo para tratar datas inexistentes.	Simplicidade e evitar uso de bibliotecas externas.
Uso de constante global para converter minutos para horas.	Uso de função para converter minutos para horas.	A conversão é uma única multiplicação, evitando definir uma função ou método da classe exclusivamente para conversão.
Iterar cada argumento da função pública gerando erro quando um é nulo.	Detectar genericamente se algum dos argumentos é nulo com any().	Permite especificar qual o argumento detectado como nulo.

4. Mudanças de comportamento

Entrada ou situação	O que acontecia antes	O que acontece agora
Argumentos nulos.	Código executava normalmente.	Exceção informa que o campo deve ser preenchido.
Data em formato incorreto.	Data era aceita.	Exceção informa que o formato é AAAA-MM-DD.
Atendimento após às 18h.	O código executa sem avisar erro.	O código gera exceção.
Último atendimento após as 18h.	Código executava normalmente.	Válida a relação horário e duração, gerando exceção.
Duração deve ser múltiplo de 15.	Duração era aceita.	O código gera exceção.

5. Como executar e gerar a documentação

Item	Resposta
Comando para compilar e executar	python ./meu_projeto/agenda_base.py

Comando para gerar a documentação de API	sphinx-apidoc -o ./docs/source/ -f ./meu_projeto sphinx-build -b markdown ./docs/source ./docs/build/markdown
Onde a documentação gerada ficou no repositório	‘docs/build/markdown’
Saída do programa de demonstração (cole aqui)	Atendimento adicionado para Ana Horario ocupado Atendimento adicionado para Carla Remarcado 14 Carla Dra. Souza MARCADO

6. Uso de ferramentas de IA

Pergunta	Resposta
Foram usadas ferramentas de IA neste trabalho?	( ) Não (X) Sim
Em quais etapas, e para quê?	O docstring no estilo Google (Sphinx) foi feito com auxílio de ferramenta de IA (ChatGPT) e ajustado pelo autor. Erros de escrita do código foram depurados com IA (ChatGPT).

Conferência antes de enviar

- ☐ O programa de demonstração compila e roda, e imprime a mesma saída do código-base.
- ☐ Toda anomalia da seção 1 aparece na seção 2, resolvida ou declarada como dívida.
- ☐ O README permite que outra pessoa execute o projeto sem perguntar nada.
- ☐ Os textos em itálico deste modelo foram apagados.